

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 7 – JUnit Fundamentos

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ♦ Verificar vs validar
 - ♦ Introducción a JUnit 5
 - ♦ Anotaciones de JUnit5
 - ♦ Estructura AAA (Arrange – Act – Assert)
 - ♦ Asserts y verificaciones automáticas
-  Práctica: unit tests sobre funciones en Java

Verificar vs Validar

✓ Validar

Proceso para confirmar que algo cumple una condición, **ya sea manual o automático**.

Ejemplos:

- Validar visualmente que un botón tiene el color correcto
- Validar que un usuario puede navegar entre pantallas

✓ Verificar

Confirmación **automática** mediante asserts o scripts.

```
assertEquals("Login exitoso", mensaje);  
assertTrue(usuario.esActivo());
```

➔ En pruebas automatizadas trabajamos mayormente con verificación.

¿Qué es JUnit?

✓ Framework estándar para pruebas unitarias en Java

JUnit permite automatizar:

- Validación de funciones
- Verificación de clases
- Pruebas de lógica de negocio
- Preparación de datos
- Integración con Selenium



Dependencia

```
<!-- https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>6.0.1</version>
  <scope>test</scope>
</dependency>
```

Estructura básica de un test con JUnit 5

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculadoraTest {

    @Test
    public void sumarTest() {
        int resultado = 2 + 3;
        assertEquals(5, resultado);
    }
}
```

✓ @Test

Indica que el método es un caso de prueba.

✓ assertEquals(esperado, real)

Compara valores y marca el test como fallido si no coinciden.

Ciclo de vida de JUnit

JUnit ejecuta los tests siguiendo este flujo:

@BeforeAll



@BeforeEach



@Test



@AfterEach



@AfterAll

✓ Ideal para entender dónde preparar entornos y dónde limpiarlos

Anotaciones principales de JUnit

@Test

Indica que un método es un caso de prueba.

```
@Test
public void sumarTest() {
    assertEquals(4, 2 + 2);
}
```

@BeforeEach

Se ejecuta **antes de cada test**.

```
@BeforeEach
public void setup() {
    calc = new Calculadora();
}
```

@AfterEach

Se ejecuta **después de cada test**.

```
@AfterEach
public void tearDown() {
    calc = null;
}
```

Anotaciones avanzadas

@BeforeAll

Corre **una sola vez** antes de todos los tests.

Requiere método **estático**.

```
@BeforeAll
static void iniciarSuite() {
    System.out.println("Iniciando suite...");
}
```

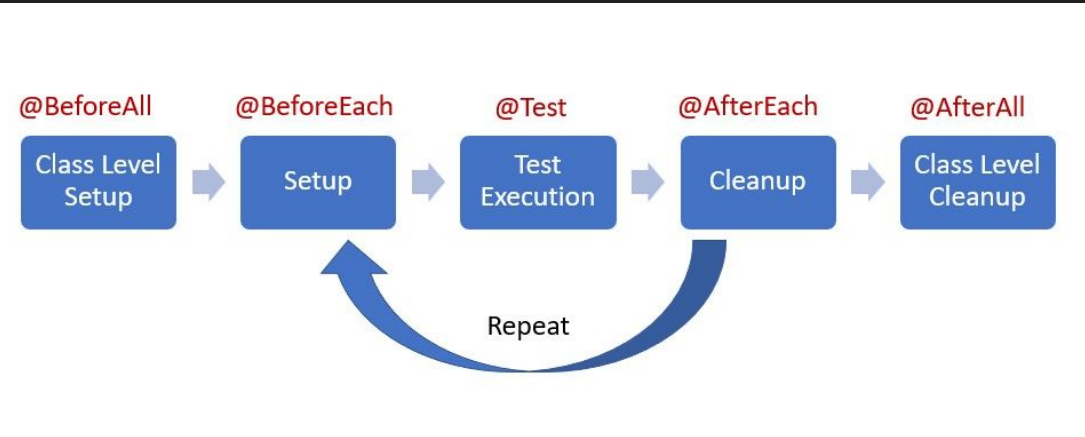
@AfterAll

Corre una sola vez al finalizar la suite.

```
@AfterAll
static void cerrarSuite() {
    System.out.println("Suite finalizada.");
}
```


¿Cuándo usar estas anotaciones?

- Preparar datos globales
- Cargar archivos de configuración
- Inicializar conexiones de base de datos
- Crear logs de inicio



Estructura general de un assert (JUnit 5)

La forma general de un assert en JUnit es:

```
assertXxx(esperado, real);  
assertXxx(condición);  
assertXxx(esperado, real, mensaje);  
assertXxx(condición, mensaje);
```

Donde:

- **esperado:** lo que debería pasar
- **real:** lo que devolvió el código
- **condición:** expresión booleana
- **mensaje:** texto opcional para entender mejor el fallo

Estructura AAA (Arrange – Act – Assert)

La estructura **AAA** es un patrón ampliamente usado para **organizar casos de prueba de manera clara y entendible**, tanto en pruebas unitarias como en pruebas automatizadas con Selenium.

Su objetivo es que cualquier persona pueda **leer un test como si fuera una historia**, entendiendo:

1. qué se prepara
2. qué se ejecuta
3. qué se verifica

```
@Test
void sumarDosNumeros() {

    // Arrange
    Calculadora calc = new Calculadora();

    // Act
    int resultado = calc.sumar(2, 3);

    // Assert
    assertEquals(5, resultado, "La suma de 2 y 3 debe retornar 5");
}
```

¿Qué significa AAA?

A – Arrange (Preparar)

Se preparan los datos, objetos, precondiciones y estados necesarios para la prueba.

Incluye:

- instanciar clases
- crear datos de prueba
- configurar mocks (si existieran)
- inicializar variables

```
// Arrange
Calculadora calc = new Calculadora();
```

A – Act (Actuar)

Es la acción que se quiere probar.

Debe ser **una sola acción principal** cuando es posible.

```
// Act
int resultado = calc.sumar(2, 3);
```

A – Assert (Afirmar / Verificar)

Es la verificación del resultado esperado.

```
// Assert
assertEquals(5, resultado, "La suma de 2 y 3 debe retornar 5");
```

Asserts (verificaciones)

Assert	Uso
<code>assertEquals(a, b)</code>	comparar valores
<code>assertTrue(cond)</code>	condición verdadera
<code>assertFalse(cond)</code>	condición falsa
<code>assertNotNull(obj)</code>	no debe ser null
<code>assertNull(obj)</code>	debe ser null
<code>assertThrows(Exception.class, () -> ...)</code>	validar excepciones

Práctica

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Ejercicio guiado

Objetivo: tester la siguiente clase

```
public class Validador {  
    public boolean esMayor(int edad) {  
        return edad >= 18;  
    }  
  
    public boolean emailValido(String email) {  
        return email.contains("@") && email.contains(".");  
    }  
}
```

- testEdadMayor()
- testEdadMenor()
- emailCorrecto()
- emailIncorrecto()

Cierre

**COLEGIO
UNIVERSITARIO**



CCC

Centro de Capacitación
Continua

Buenas prácticas

Usar la estructura AAA (Arrange – Act – Assert)

La regla fundamental para escribir tests claros y profesionales.

✓ Separar claramente las secciones:

- **Arrange** → Preparar datos, objetos, precondiciones
- **Act** → Ejecutar la acción a probar
- **Assert** → Verificar el resultado

✓ Comentar siempre cada sección para mejorar la legibilidad.

✓ Mantener **un solo Act principal** por test.

Escribir asserts claros, descriptivos y alineados al objetivo del test

- ✓ Siempre indicar **qué se está verificando**
- ✓ Usar el mensaje opcional del assert para explicar el motivo del fallo
- ✓ Usar el assert adecuado (no usar equals con null, sino assertNull)
- ✓ No poner lógica compleja dentro del assert

Diferenciar entre verificar y validar

- ✓ **Verificar** → automatizado con asserts
- ✓ **Validar** → puede ser manual / visual

En automatización se prioriza **verificar**, por lo que cada prueba debe incluir al menos un assert significativo.

Aplicar el ciclo de vida JUnit para preparar y limpiar pruebas

- ✓ Usar `@BeforeEach` para inicializar objetos antes de cada test
- ✓ Usar `@AfterEach` para liberar recursos
- ✓ Usar `@BeforeAll` y `@AfterAll` para tareas globales

Esto garantiza que los tests no dependan uno de otro y evita efectos colaterales.

Escribir tests pequeños, independientes y legibles

- ✓ Un test = un comportamiento
- ✓ No mezclar varios escenarios en un mismo test
- ✓ Evitar dependencias entre tests
- ✓ No usar valores “mágicos”; nombrar lo que se está probando

```
✗ assertEquals(82, valor)
✓ assertEquals(82, valorCalculado, "El valor calculado no es el esperado")
```

Usar nombres descriptivos para los métodos de test

Un test debe leerse como una frase.

```
sumar_deberiaRetornar5_cuandoSeSuma2y3()
login_deberiaFallar_cuandoPasswordEsIncorrecto()
email_deberiaSerInvalido_sinCaracterArroba()
```

Mantener los tests rápidos y deterministas

- ✓ Sin sleeps innecesarios
- ✓ Sin dependencias externas no controladas
- ✓ Sin aleatoriedad sin control

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua